Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Master's thesis

INTERPRETABLE METHODS FOR DATA-TO-TEXT GENERATION

Dominik Maćkowiak, 151915

Supervisor
Mateusz Lango, PhD

Poznań, 2026

Abstract

Contents

Chapter 1

Introduction

1.1 Motivation

1.2 Thesis structure

The structure of the thesis is as follows. Chapter 2 provides an overview of data-to-text generation. Chapter 3 describes the methods for building interpretable meaning representations from complex data. Chapter 4 presents the surface realization approach using OpenEvolve. Chapter 5 reports the experimental results. Chapter 6 discusses the findings, and Chapter 7 concludes the thesis.

Chapter 2

Data-to-Text Generation

2.1 Problem definition

2.2 Methods

2.3 Interpretable D2T Approaches

2.4 Performance Evaluation

Chapter 3

Building interpretable meaning representations from complex data

3.1 Method Overview

3.2 Data conversion with LLMs

Input data format. All three methods operate on a collection of *raw structured instances*. A raw structured instance is a single JSON object representing one self-contained data record, such as a weather forecast for a specific city, a product description, or a sports event summary. Each instance contains nested fields with structured data (e.g., time series, categorical values, numerical measurements) that must be converted into semantic triples. For example, a weather forecast instance might contain fields for city name, date, temperature values, wind speed, and weather conditions. Let $D = \{d_1, \ldots, d_N\}$ denote the set of N such instances extracted from the input JSON.

3.2.1 Text-First Extraction with Normalization

Text-First Extraction with Normalization decomposes the data-to-triples problem into two fully batched LLM stages and a normalization stage. The method deliberately avoids any inter-instance coupling during extraction: each reference and each triple set is produced independently and in parallel, which makes the method stateless, at the cost of not enforcing predicate consistency during extraction.

Stage 1 — Reference text generation. In the first stage the model is invoked once per instance under a `text-generation system prompt`, P_{txt} (see Appendix A.1), whose objective is to convert one structured data instance into concise natural language and to return the result as a JSON object of schema `{"text": "..."}`.[1] The prompt further instructs the model to capture the most important information, trends, extremes, and notable conditions, and to summarize time series (e.g., weather forecasts) at a high level rather than listing every data point.

All N requests are packaged as a single OpenAI Batch API job and a single request outputs t_i, where t_i is the generated reference text for instance d_i. This batched stage yields, for every instance, a natural-language reference t_i that constitutes the intermediate representation from which triples are subsequently derived.

[1] Throughout this work, JSON schemas are presented in simplified notation with placeholder values (e.g., `"text": "..."`) for readability, rather than as formal JSON Schema definitions.

Stage 2 — Triples-from-text extraction. A second OpenAI Batch is then assembled in which every reference t_i is paired with a `triples-from-text system prompt`, P_{tr} (see Appendix A.1), whose objective is to extract semantic triples from natural language text, returning JSON of schema `{"triples":["subject":"...","predicate":"...","object":"..."]}`. The prompt imposes two constraints: it instructs the model to preserve the full meaning of the input text and to keep predicates in `lower_snake_case` when possible, and to avoid duplicate triples. Each instance is therefore an independent text-to-triples problem, and the batch can be fully parallelized. The output of a single request is T_i, where T_i is a set of triples that describe the reference text t_i.

Stage 3 — Post-hoc predicate normalization. Because Stage 2 imposes no inter-instance vocabulary control, this stage invokes a predicate-normalization routine. The routine operates on the flattened list of triples and iteratively merges semantically equivalent predicates:

Let Π be the set of distinct predicates present in the extracted triples. A Union-Find structure over Π is initialized with every predicate in its own singleton class. A *Union-Find* (disjoint-set) is a data structure that maintains a collection of disjoint sets and supports two operations: `find(x)` returning the representative element of the set containing x, and `union(a, b)` merging the sets containing a and b. A *singleton class* is an equivalence class containing only a single element.

For each round $r = 1, 2, \ldots$ (bounded by $2|\Pi|$):

- The model is prompted with the current set of canonical predicates (class representatives) and asked to propose candidate equivalent pairs under a `candidate-pairs system prompt` (see Appendix A.1).
- If no candidate pairs are returned, or if the candidate set is identical to one proposed in an earlier round (indicating the model has converged or is cycling without progress), the loop terminates.
- Otherwise, every candidate pair is sent to the model under a `strict-equivalence system prompt` (see Appendix A.1) that returns a JSON object with three fields: `equivalent` (boolean), `confidence` (a score between 0 and 1), and `reason` (a textual explanation). The model is instructed to judge equivalence strictly (mergeable in a knowledge graph), not mere relatedness.
- Pairs judged equivalent are merged in the Union-Find structure. If a round produces no merges, the loop terminates.

After termination, each predicate is mapped to the canonical representative of its class, defined as the first member of the class (deterministic, order-based choice). The triples are then rewritten with these canonical predicates.

In the normalization step, the final output combines: (i) generated references, (ii) extracted triples, and (iii) normalized triples alongside the canonical mapping and the comparison audit trail.

3.2.2 Blueprint-Iterative Refinement

Blueprint-Iterative Refinement introduces an explicit notion of extraction blueprints that act as an intermediate, human-inspectable artifact between the domain declaration and the final triple extraction. An *extraction blueprint* is a structured specification containing predicate labels, example triples, example text, and transformation guidelines. The blueprints are generated top-down from the domain name, then iteratively validated and revised against the generated references

until it is confirmed that they can extract triples for every instance. Only then is a single batched triple-extraction run executed under the finalized blueprint.

Stage 1 — Initial blueprint generation. Given the user-supplied problem domain string δ (e.g., weather forecast), the model is invoked once under a `blueprint-creation system prompt`, $P_{\text{blueprint}}$ (see Appendix A.2), and asked to design an extraction blueprint for a domain. Its response is constrained by a strict JSON schema to contain exactly four fields:

$$R = \Big(\Pi_R,\ E,\ T_{\text{ex}},\ G\Big),$$

where Π_R is a list of predicate labels, E is a list of example triples $\{(\sigma_k, \pi_k, \omega_k)\}$ using those predicates, T_{ex} is a short natural-language passage consistent with the example triples, and G is a list of free-text transformation guidelines. The response is sanitized, which deduplicates predicates and guidelines, discards malformed example triples, and falls back to the corresponding field values from the previous blueprint R_{r-1} for any field that is empty or malformed (for the initial blueprint R_0, empty fields default to empty lists or strings). The resulting R_0 constitutes the initial blueprint. Conceptually, R_0 is a self-contained specification of how the model believes instances of the domain should be verbalized and triplelized, complete with concrete formatting conventions embedded in the example triples.

Stage 2 — Reference text generation. The reference-generation batch is identical to that of the previous method: every instance d_i is sent to the `text-generation system prompt` P_{txt} within a single OpenAI Batch, producing references t_i.

Stage 3 — Blueprint validation and iterative revision. The core of the method is an iterative loop, bounded by $R_{\max}$. The loop maintains a current blueprint R and operates as follows for each round $r = 1, \ldots, R_{\max}$:

1. **Feasibility scan.** A random reference text with replacement t_i is selected and the model is invoked under a `blueprint-feasibility system prompt`, P_{feas} (see Appendix A.2), which receives the current blueprint R and the text t_i, and returns a JSON object with three fields: `possible` (boolean indicating whether the blueprint is sufficient), `reason` (textual explanation), and `blueprint_gaps` (a list of descriptions of what is missing from the blueprint in order to extract triples from t_i).

2. **Pass condition.** If the scan reaches the $R_{\max}$ without encountering a `possible = false` judgment, the blueprints are declared validated against all texts and the loop terminates.

3. **Failure and revision.** As soon as a reference t_{i^*} is judged infeasible under R, the scan halts. The `blueprint-revision system prompt`, P_{rev} (see Appendix A.2), is then invoked with the domain δ, the current blueprint R, the failing text t_{i^*}, the failure reason, and the enumerated `blueprint_gaps`. Its response is again constrained to the same four-field schema structure $R = (\Pi_R, E, T_{\text{ex}}, G)$ described in Stage 1. The model is instructed to keep existing high-quality blueprint components, but add or adjust what is necessary to cover the failing case. The sanitized result replaces R as the new current blueprint.

4. **Restart.** Because revising the blueprint may break previously validated instances or expose new gaps, the scan is restarted and the round number r set to 1. The loop thus monotonically accumulates coverage, but backtracks the scan position whenever the blueprint is modified.

If the budget $R_{\max}$ is exhausted before all texts are validated, the method emits a warning and proceeds to final extraction with the best-effort blueprint; the output records that validation was incomplete.

This stage implements a generate-test-revise loop in which the blueprint is the artifact being refined, the reference texts act as test cases, and the feasibility prompt acts as a model-internal test oracle.

Stage 4 — Final batched extraction under finalized blueprint. Once the validation loop has terminated, a single OpenAI Batch is submitted in which every reference t_i is paired with a `triples-from-text-with-blueprint system prompt`, $P_{\text{tr\&}R}$ (see Appendix A.2). This prompt instructs the model to extract semantic triples from the natural language text using the provided blueprint, to follow the blueprint guidelines, to mimic the formatting style of the example triples, and to prefer predicates from the blueprint predicate list whenever possible. The blueprint R is serialized and injected in each request. The batch output is parsed into T_i.

3.2.3 Evolving Catalog

Evolving Catalog is a hybrid text-first data-to-triples pipeline that combines an evolving predicate catalog with a stability-triggered transition from sequential to batch processing. It is designed to keep the predicate vocabulary consistent across instances while reducing the latency overhead of per-instance, order-dependent LLM calls.

Stage 1 — Reference text generation. An OpenAI Batch is assembled in which every instance d_i is paired with the `text-generation system prompt` P_{txt} (see Appendix A.3), whose purpose is to produce a concise natural-language summary that captures the most important information, trends, extremes, and notable conditions, and summarizes time series at a high level. The batch output is parsed into text reference t_i.

Stage 2 — Sequential catalog-driven triple extraction. The second stage maintains a mutable predicate catalog C, initialized as empty. The catalog is a set of entries $\{(\pi, e_\pi)\}$, where each entry associates a predicate label π with an example triple $e_\pi = (s, \pi, o)$ in which that predicate was first observed. The catalog thus carries both a normalized vocabulary and a small set of usage examples that serve as in-context guidance for the extraction model.

The instances are processed strictly in order:

1. **Bootstrap instance ($i = 0$).** Because C is empty, the reference t_0 is sent to the model under a `first-instance system prompt`, P_{first} (see Appendix A.3), that simply asks for semantic triples from the text with concise `lower_snake_case` predicates and no duplicates. This yields a set of triples T_0.

2. **Catalog-guided instances ($i \geq 1$).** For each subsequent instance d_i, the reference t_i is sent under a `catalog-guided system prompt`, P_{cat} (see Appendix A.3), in which the current catalog C is serialized and injected into the user prompt. The model is instructed to use only predicates from the provided catalog when possible and to introduce a new predicate only when none of the catalog predicates can describe the fact. This yields triples T_i.

   After each instance, every triple in T_i is inspected. If a triple contains a predicate $\pi \notin C$, a new entry (π, e_π) is added to C, where e_π is set to that triple (the first occurrence).

Stage 3 — Stable-window heuristic and batched tail processing. A counter w tracks the number of consecutive instances that did not introduce any new predicate; it is reset to zero whenever a new predicate is added. Let W denote the stable window parameter. The sequential phase terminates as soon as $w \geq W$ and at least one instance remains unprocessed. Let i^* be the index at which this condition is first met; the catalog C at that point is declared frozen.

All instances $d_{i^*+1}, \ldots, d_N$ (the tail) are then processed in a single second OpenAI Batch, again under P_{cat} and with the frozen catalog C injected into every request. Two design choices are important here:

- The tail is parallelized, hence the cost of the remaining $N - i^* - 1$ LLM calls is amortized into one batch.
- Predicates that first appear within the tail are not added back to C. The catalog is finalized at the moment of stability, which guarantees that the predicate vocabulary presented to a human evaluator is fully determined by the sequential phase and does not silently grow during batch processing.

3.3 Fine-tuned model for data conversion

The methods described in Section 3.2 rely on repeated invocations of a general-purpose large language model at inference time: each instance requires one or more LLM calls to produce reference text, extract triples, and, in some pipelines, to validate or revise intermediate artifacts. While this design keeps the pipeline flexible, it incurs substantial latency and computational cost, particularly when the same domain is processed repeatedly. An alternative is to *distill* the behaviour of the multi-stage pipeline into a single, domain-specialized model that produces both the natural-language reference and the corresponding triple set in one decode pass. This section describes how such a model is obtained via supervised fine-tuning with QLoRA.

3.3.1 Supervised Fine-Tuning and QLoRA

Supervised fine-tuning (SFT) adapts a pre-trained language model to a specific task by training it on labeled input–output pairs. Given a pre-trained model M_θ with parameters θ and a dataset $\{(x_i, y_i)\}_{i=1}^{n}$ of input–output examples, SFT minimizes the negative log-likelihood of the targets [5]:

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{t=1}^{|y_i|} \log P_\theta(y_{i,t} \mid x_i, y_{i,<t}). \tag{3.1}$$

For small models (up to a few billion parameters), full SFT—updating all parameters θ—is feasible on modern GPU hardware. However, for models with tens of billions of parameters, full fine-tuning becomes prohibitively expensive: a 31-billion-parameter model in 16-bit precision requires approximately 62 GB of memory for the weights alone, and the Adam optimizer [3] maintains two additional moment estimates per parameter, roughly doubling the memory footprint.

Parameter-efficient fine-tuning (PEFT) methods address this by keeping the base model weights frozen and introducing a small number of trainable parameters [4]. Among these, *Low-Rank Adaptation* (LoRA) [2] has become the most widely adopted. LoRA decomposes the weight update into a low-rank product: for each frozen weight matrix W_0, the forward pass is modified as

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x, \tag{3.2}$$

where:

- $W_0 \in \mathbb{R}^{d \times k}$ is the frozen pretrained weight matrix
- d is the output dimension and k is the input dimension of the layer
- $x \in \mathbb{R}^k$ is the input vector to the layer

- $h \in \mathbb{R}^d$ is the output (hidden state) of the layer
- $A \in \mathbb{R}^{r \times k}$ is the first trainable low-rank matrix (down-projection)
- $B \in \mathbb{R}^{d \times r}$ is the second trainable low-rank matrix (up-projection)
- $r \ll \min(d, k)$ is the rank of the decomposition
- α is a scaling factor that controls the magnitude of the low-rank update

Only A and B are updated by the optimizer; W_0 remains frozen and requires no gradient storage or optimizer state. For a typical transformer with attention projections W_q, W_k, W_v, W_o and feed-forward projections $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$, LoRA adapters are attached to each projection, yielding a trainable parameter count that is a small fraction of the base model (on the order of tens of millions versus tens of billions).

QLoRA [1] extends LoRA with two memory-reduction techniques that together allow fine-tuning a 31-billion-parameter model on a single GPU with 40 GB of VRAM. First, the frozen base model weights are quantized to *4-bit NormalFloat* (NF4), a quantization datatype introduced by Dettmers et al. [1] that is information-theoretically optimal for normally distributed weights (see Section 3 of their paper for theoretical justification). A *double quantization* step further quantizes the quantization constants themselves, reducing the per-parameter overhead. Second, *paged optimizers* leverage NVIDIA unified memory to transparently page optimizer state between GPU and CPU memory when the GPU reaches capacity, avoiding out-of-memory failures during gradient accumulation steps. The forward and backward passes are performed in `bfloat16` precision: the 4-bit weights are dequantized on the fly for each matrix multiplication, preserving numerical quality close to a full-precision fine-tune while keeping the memory footprint low.

At the data scales typical for domain-specific data-to-text tasks (on the order of several hundred to a few thousand labeled examples), LoRA's low-rank constraint also acts as a regularizer: by limiting the number of trainable parameters, it reduces the risk of overfitting and preserves the general linguistic capabilities acquired during pre-training [2].

3.3.2 Training Data Construction

Rather than collecting human-annotated text–triple pairs, the training data for the fine-tuned model is produced by *distilling* the outputs of the tripler pipeline described in Section 3.2. For each raw structured instance d_i processed by a tripler method, the pipeline already produces a reference text t_i and a set of normalized triples $T_i = \{(s_k, \pi_k, o_k)\}$. These two outputs, aligned by instance identifier, constitute exactly the joint target that the fine-tuned model should learn to produce in a single pass.

Each training example is formatted as a three-message conversation following the base model's chat template:

- **System prompt.** An instruction directing the model to convert one structured data instance into concise natural language and a corresponding set of RDF semantic triples (see Appendix B for the complete prompt text), and to return the result as a JSON object of schema `{"text": "...", "triples": [{"subject": "...", "predicate": "...", "object": "..."}]}`.
- **User message.** A prompt that presents the serialized instance and requests the joint output (see Appendix B for the complete template). The wording and instance serialization are kept

byte-identical to the prompts used by the tripler pipeline at inference time, ensuring *prompt parity*: the token sequence seen during training matches the token sequence the model will encounter when served.

- **Assistant message (target).** A JSON string encoding the reference text and the normalized triples (see Appendix B for the expected output format): `{"text":` t_i`, "triples":` T_i`}`.

The messages are rendered through the base tokenizer's `chat_template` function, which applies the special tokens and formatting expected by the model architecture. This ensures that the token sequences fed to the trainer are identical to those assembled by the serving engine at inference time.

A notable consequence of this distillation approach is that the fine-tuned model is bounded by the quality of the reference labels: it can learn to reproduce the pipeline's outputs more efficiently and consistently, but it cannot exceed their quality. Improving the reference labels is an upstream concern addressed by the tripler pipeline design, not by fine-tuning.

The dataset is split into a training, evaluation and test set. The split is deterministic, controlled by a fixed random seed, so that results are reproducible across runs. One fine-tuned model is produced per domain (mobile phone specifications, weather forecasts, Wikidata entries, illness reports), keeping the predicate vocabulary and verbalization style specialized to that domain.

3.3.3 Training Configuration

The base model for all fine-tuning experiments is *Gemma 4 31B IT* [6][7], a 31-billion-parameter instruction-tuned language model released by Google. Gemma 4 uses a hybrid attention architecture: most layers employ sliding-window attention with head dimension 256, while a subset of layers use global attention with head dimension 512. The model is loaded with the `SDPA` (Scaled Dot-Product Attention) implementation, which automatically dispatches each layer to a compatible backend, accommodating the mixed head dimensions without requiring per-layer patching.

All training is performed on NVIDIA A100-SXM4 GPUs with 40 GB of VRAM, using the PLGrid Athena cluster. Table 3.1 summarizes the complete training configuration.

Table 3.1: QLoRA training configuration

Component	**Value**
Base model	Gemma 4 31B IT
LoRA rank (r)	16
LoRA scaling (α)	32
LoRA dropout	0.05
Target modules	420 (all attention and MLP projections)
Trainable parameters	25M (<0.1% of base model)
Quantization	4-bit NF4 with double quantization
Compute precision	bfloat16
Optimizer	Paged AdamW 8-bit
Learning rate	10^{-4} (cosine schedule)
Warmup ratio	0.03
Epochs	3
Effective batch size	16 (per-device: 1, accumulation: 16)
Max sequence length	4096–8192 (domain-dependent)
Loss type	Completion-only (prompt masked)

The LoRA configuration targets all attention and MLP projection matrices in the language model backbone (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`), using a

regular expression that scopes the adapter placement to the language model sub-network and excludes the vision and audio towers present in the multimodal Gemma 4 checkpoint. This yields approximately 420 target modules and roughly 25 million trainable parameters—less than 0.1% of the base model.

The maximum sequence length is adjusted per domain to accommodate the varying sizes of the raw structured instances while staying within the GPU memory budget. Wikidata instances are small (at most a few hundred bytes), so a maximum length of 8192 tokens is used. GSM Arena and OWID instances are larger, and a maximum length of 4096 tokens covers the majority of examples. OpenWeather forecast instances are the largest (approximately 18 KB each), and a maximum length of 6144 tokens is used to retain the bulk of each example while remaining within the activation memory ceiling. Instances exceeding the maximum length are truncated.

After training, the LoRA adapter weights are combined with the frozen base model weights to produce a single, standalone model. This process, known as *weight merging*, computes $W_{\text{final}} = W_0 + \Delta W$ for each adapted layer, effectively absorbing the adapter's learned modifications into the base model. The resulting model contains no separate adapter components and can be served as a standard `bfloat16` checkpoint. The merged model is a drop-in replacement for the base model: it can be served by the same vLLM inference engine used by the tripler pipeline, with no adapter-loading overhead at inference time. The result is a domain-specialized model that, given a raw structured instance, produces both a natural-language verbalization and the corresponding set of semantic triples in a single decode pass.

Chapter 4

Surface realization — evolving programs using AlphaEvolve

Chapter 5

Experiments

5.1 Meaning representation

5.2 Surface realization

5.3 Whole system evaluation

Chapter 6

Discussion

Chapter 7

Summary

Bibliography

[1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. https://arxiv.org/abs/2305.14314, 2023.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. https://arxiv.org/abs/2106.09685, 2021.

[3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. https://arxiv.org/abs/1412.6980, 2014.

[4] Sourab Mangrulkar, Sylvain Gugger, Lysak Lysak, et al. PEFT: Parameter-efficient fine-tuning of billion-scale models on low-resource hardware. https://github.com/huggingface/peft, 2022.

[5] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Ray, et al. Training language models to follow instructions with human feedback. https://arxiv.org/abs/2203.02155, 2022.

[6] Gemma Team et al. Gemma: Open models based on gemini research and technology. https://arxiv.org/abs/2403.08295, 2024.

[7] Gemma Team et al. Gemma 4 technical report. https://arxiv.org/abs/2607.02770, 2026.

Appendix A

Prompts

A.1 Text-First Extraction with Normalization

This section contains the complete text of all system prompts used in the Text-First Extraction with Normalization method (Section 3.2).

Text-Generation System Prompt (P_{txt})

> You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."}. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Triples-From-Text System Prompt (P_{tr})

> You extract semantic triples from natural language text. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...","predicate":"...","object":"..."}]}. Rules: preserve the full meaning of the input text, avoid duplicate triples, and keep predicates in lower_snake_case when possible.

Candidate-Pairs System Prompt

> You find candidate predicate pairs that may be semantically equivalent. Return ONLY JSON with schema: {"pairs":[{"predicate_a":"...","predicate_b":"..."}]}. Rules: include only pairs with high likelihood of strict semantic equivalence; do not include pairs that are merely related; do not include duplicates or self-pairs.

Strict-Equivalence System Prompt

> You decide if two predicates are semantically equivalent in meaning. Return ONLY JSON with schema: {"equivalent": true|false, "confidence": 0.0-1.0, "reason": "..."}. Be strict: equivalent only if they can be safely merged in a knowledge graph.

A.2 Blueprint-Iterative Refinement

This section contains the complete text of all system prompts used in the Blueprint-Iterative Refinement method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."}. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Blueprint-Creation System Prompt ($P_{\text{blueprint}}$)

You design extraction rules for a domain. Return ONLY JSON with schema: {"predicates":["..."], "example_triples":[{"subject":"...","predicate":"...","object":"..."}], "example_text":"...", "guidelines":["..."]}. Rules: include precise and reusable predicates, provide concrete example triples with strict formatting, provide an example text matching those triples, and give practical transformation guidelines.

Blueprint-Feasibility System Prompt (P_{feas})

You evaluate whether given extraction rules are sufficient to extract triples from a text. Return ONLY JSON with schema: {"possible":true|false,"reason":"...","rule_gaps":["..."]}.

Blueprint-Revision System Prompt (P_{rev})

You revise extraction rules so triples can be produced from a text. Return ONLY JSON with schema: {"predicates":["..."], "example_triples":[{"subject":"...","predicate":"...","object":"..."}], "example_text":"...", "guidelines":["..."]}. Keep existing high-quality rules, but add or adjust what is necessary to cover the failing case.

Triples-From-Text-With-Blueprint System Prompt ($P_{\text{tr\&}R}$)

You extract semantic triples from natural language text using provided rules. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...","predicate":"...","object":"..."}]}. Follow rule guidelines and mimic the formatting style from example triples. Prefer predicates from the rule predicate list whenever possible.

A.3 Evolving Catalog

This section contains the complete text of all system prompts used in the Evolving Catalog method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."}. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

First-Instance System Prompt (P_{first})

You extract semantic triples from text. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...","predicate":"...","object":"..."}]}. Use concise predicate labels in lower_snake_case when possible and avoid duplicates.

Catalog-Guided System Prompt (P_{cat})

You extract semantic triples from text using a predicate catalog with examples. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...","predicate":"...","object":"..."}]}. Use only predicates from the provided catalog when possible. Introduce a new predicate only when none of the catalog predicates can describe the fact.

Appendix B

Prompts for Fine-tuned Model

System Prompt for Joint Text and Triple Generation

You convert one structured data instance into concise natural language and a corresponding set of RDF semantic triples. Return ONLY JSON with schema: {"text":"...","triples":[{"subject":"..
Capture the most important information, trends, extremes, and notable conditions. Use concise predicate labels in lower_snake_case when possible and avoid duplicates. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

User Message Template

Create a concise natural-language summary of this ONE data instance and extract its semantic triples.

instance_context=`{serialized JSON instance}`

Return JSON only.

Assistant Message (Expected Output)

{"text": "<generated natural language text>", "triples": [{"subject": "...", "predicate": "...", "object": "..."}, ...]}

Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Typeset using LaTeX in Computer Modern.

BibTeX:

```
@mastersthesis{ key,
    author = "Dominik Maćkowiak",
    title = "{Interpretable methods for data-to-text generation}",
    school = "Poznan University of Technology",
    address = "Pozna{\'n}, Poland",
    year = "2026",
}
```